

coverage-matrix

Test-Type Coverage Matrix (Living Ledger)

Field	Value
Revision	1
Last modified	2026-06-10
Status	LIVING LEDGER — current state verified against the tree on 2026-06-10; targets are PLANNED (SP-owned)
Scope	helix_code/ (inner Go app dev.helix.code) + delegated submodules/helix_qa + submodules/challenges
Source plan	docs/superpowers/specs/plans/2026-06-10-SP7-testing-qa-plan.md (§1 ground truth, §2 matrix, §7 guard rows)
Authority	Cascades from constitution/ + root CLAUDE.md/CONSTITUTION.md. Anti-bluff §11.4 governs every PASS.

Anti-bluff note (§11.4 / §11.4.123). This ledger separates **what exists today** (file:line or ABSENT, verified against the live tree on 2026-06-10) from **the target** (PLANNED, owned by a sub-program). A cell reading ABSENT or THIN is an honest coverage gap, not a failure to find. CONST-050(B): a *delegated* HelixQA/Challenge slot counts **ONLY** if it truly exercises HelixCode with captured evidence — otherwise it is a coverage-bluff risk, flagged below.

1. How to read this ledger

- **Rows** = the 15 supported test types (plus two cross-cutting meta rows: §1.1 paired-mutation, regression-guard).
- **Columns:**
 - **Today’s status** — file:line / dir / if present; ABSENT / THIN / DELEGATED otherwise (verified 2026-06-10).
 - **Target** — the intended end state per the SP7 plan.
 - **Owning SP** — which sub-program’s feature area the type must cover.
- **SP feature areas** (the 6 columns of “what must be covered”): | SP | Feature area | |——|
———| | **SP1** | Model access (key recognition → working-model funnel) | | **SP2** | HelixAgent exposure (unified catalog, per-provider/per-model targeting) | | **SP4** | CLI bridge (spawn real CLI agents; prompt→edit→commit) | | **SP5** | Parallel / subagent execution (worker-pool, scheduler, consensus) | | **SP6** | Dynamic flows (flow engine / DAG / pipeline / agent-mesh) | | **SP7** | Cross-cutting QA (this ledger; autonomous sessions; guard registry) |

Verified ground truth (SP7 §1, re-confirmed 2026-06-10): HelixCode-local coverage is **STRONG** for unit / integration / e2e / security / chaos / stress / performance / benchmarking, with a real §1.1 meta-test harness at `helix_code/tests/stresschaos/stresschaos_meta_test.go`. The gaps are concentrated in ddos / scaling / ux / ui (local), HelixQA staleness, and the per-defect regression-guard registry.

2. The 15 test types × current status × target × owning SP

#	Test type	Today’s status (helix_code-local + delegated)
1	unit	PRESENT — <code>helix_code/internal/**/*_test.go</code> , <code>helix_c</code>
2	integration (-tags=integration)	PRESENT — <code>helix_code/tests/integration/ (api_integr approval_test.go); test-integration-full:222</code>
3	e2e	PRESENT — <code>helix_code/tests/e2e/ (complete_workflow_ comprehensive_matrix_test.go); test-e2e-full:228</code>
4	full-automation	DELEGATED + THIN local — <code>HelixQA cmd/helixqa autonomous automation/; test-complete:246</code>
5	security	PRESENT — <code>helix_code/tests/security/ {owasp, authentication, authorization, tools_security scan*:751-775; test-security-full:234</code>
6	ddos	ABSENT local (<code>helix_code/tests/ddos</code> does not exist) — <code>DELEO scripts/ddos_health_flood_challenge.sh + banks/ddos</code>
7	scaling	ABSENT local (<code>helix_code/tests/scaling</code> does not exist) — <code>DE scripts/scaling_horizontal_challenge.sh</code>

#	Test type	Today's status (helix_code-local + delegated)
8	chaos	PRESENT — helix_code/tests/stresschaos/chaos.go + pe stress-chaos:114
9	stress	PRESENT — helix_code/tests/stresschaos/stresschaos (internal/worker/worker_pool_stress_test.go, task/, a
10	performance	PRESENT — helix_code/tests/performance/ {benchmark_test.go,competitor_baseline_test.go,ppr test-benchmark:157
11	benchmarking	PRESENT — same as #10 + PGO pgo-refresh:90 + HelixQA bank
12	ui	THIN local — helix_code/applications/{terminal-ui,de ui_terminal_interaction_challenge.sh
13	ux	ABSENT local (helix_code/tests/ux does not exist) — DELEGAT scripts/ux_end_to_end_flow_challenge.sh
14	Challenges	PRESENT — submodules/challenges/{banks,challenges, helix_code/tests/e2e/challenges/
15	HelixQA banks / autonomous-QA-session	DELEGATED + STALE — HelixQA cmd/helixqa autonomous + l v4.0.0-393-g4d2dcb2 (round 219 / 2026-05-19)

Coverage gaps found: 6 — GAP-1 ddos-local, GAP-2 scaling-local, GAP-3 ui-thin(+pixel-oracle), GAP-4 ux-local, GAP-5 HelixQA-stale/no-sync-channel, GAP-6 regression-guard-registry (rows below).

3. Cross-cutting meta rows

Row	Today's status	Target (PLANNED)	Own SP
§1.1 paired-mutation meta	PRESENT — <code>helix_code/tests/stresschaos/stresschaos_meta_test.go</code> (plants deadlock/leak/error-rate/below-floor/chaos-panic, asserts detection); <code>stress-chaos-meta:121</code>	every NEW harness/gate gets its own paired mutation (each Phase A harness, every CM- * gate)	ev SP har + g
regression-guard	PRESENT-but-not-a-registry — <code>helix_code/tests/regression/{critical_paths_test.go,server_timeout_test.go}</code> (no RED_MODE polarity, no D-N rows)	GAP-6 → standing <code>helix_code/tests/regression/guards/</code> registry with RED_MODE switch (§11.4.115), tag-blocking, risk-ordered first (§11.4.132) (SP7 Phase E)	ev fix def acr all

4. Per-defect regression-guard rows (GAP-6, SP7 Phase E §7)

Each guard = one source with a RED_MODE polarity switch: RED_MODE=1 reproduces the historical defect on the pre-fix artifact (the proof the guard is real); RED_MODE=0 is the standing GREEN guard asserting absence (§11.4.115). **All ABSENT today** (`helix_code/tests/regression/guards/` does not exist).

Guard file (PLANNED)	Defect	RED (=1)
<code>D1_completion_models_guard_test.go</code>	D-1 hardcoded 3-model list — <code>helix_agent/internal/handlers/completion.go:406</code> (verified present)	<code>/v1/com</code> <code>models</code> static 3-ite
<code>D2_cli_listmodels_guard_test.go</code>	D-2 CLI lists failed/pending as available — <code>cmd/cli/main.go:1361</code> (verified present)	a failed model app handlel output
<code>D3_loadapikeys_wired_guard_test.go</code>	D-3 dead <code>secrets.LoadAPIKeys</code> — <code>internal/secrets/loader.go:30</code> (verified: no prod call-site)	call-graph LoadAPI unreferen path

Guard file (PLANNED)	Defect	RED (=1
D4_working_model_filter_guard_test.go	D-4 filter loaded-never-applied — adapter.go:175 (verified: GetMinAcceptableScore exists, GetWorkingModels ABSENT)	filter conf output inc working m
D5_provider_hardcode_guard_test.go	D-5 hardcoded lists — openai_provider.go, anthropic_provider.go	provider C returns a s slice
D6_cli_agent_exec_guard_test.go	D-6 stub returns hardcoded strings — helix_agent/.../qwencode.go:101/115/137	method re string with
D7_replace_dev_helix_agent_guard_test.go	D-7 helix_code/go.mod lacks replace dev.helix.agent	go.mod ha replace agentic importabl

5. Anti-delegation-bluff gates (SP7 Task A5 — PLANNED)

To stop a delegated HelixQA slot from being a config-only PASS (CONST-050(B)), each delegated row earns a pre-build gate that asserts the HelixQA script's last run emitted captured evidence referencing a **live** HelixCode endpoint/binary (parsing qa-results/), each with a paired §1.1 mutation:

Gate (PLANNED)	Converts row	Asserts
CM-DDOS-HITS-HELIXCODE	#6 ddos	the ddos script hit a real HelixCode endpoint with captured p50/p95/p99
CM-SCALING-HITS-HELIXCODE	#7 scaling	scale-out exercised real internal/worker with a captured throughput table
CM-UX-HITS-HELIXCODE	#13 ux	the journey drove the real CLI with a captured bidirectional transcript

Gate (PLANNED)	Converts row	Asserts
CM-UI-HITS-HELIXCODE	#12 ui	the TUI interaction asserted on real rendered cells / OCR self-validation

6. N/A or delegated-by-design (honest coverage notes, §11.4.3)

- **SP1 (server/CLI-only surface):** UI/UX/DDoS/scaling are N/A-or-delegated at the SP1 layer; SP7 owns the cross-cutting local harnesses (Phase A). Recorded as an honest coverage note, never faked (SP1 plan §7).
- **DDoS/scaling/ux** currently have NO HelixCode-local harness — the delegated HelixQA scripts are the only assets and are unproven against HelixCode endpoints until the Task A5 gates land. Treat their PASS as provisional until evidence-backed.

7. Summary

1. The 15 types split into **strong-local** (unit, integration, e2e, security, chaos, stress, performance, benchmarking) and **gap** (ddos, scaling, ux, ui-thin, HelixQA-stale) plus the regression-guard registry gap.
2. **8 types are working locally today** with a real §1.1 meta-test harness backing the stress/chaos floor.
3. **4 types have local gaps** (ddos/scaling/ux ABSENT, ui THIN) — covered today only by delegated HelixQA scripts, which are a CONST-050(B) coverage-bluff risk until the CM-* -HITS-HELIXCODE gates prove they hit real HelixCode.
4. **HelixQA is stale** (round 219 / 2026-05-19, pin v4.0.0-393-g4d2dcb2) and not wired to a §11.4.116 sync-channel — Phase B/C target.
5. **The per-defect regression-guard registry (GAP-6) is ABSENT** — D-1..D-7 guards are all PLANNED; the existing tests/regression/ files are not a RED_MODE registry.
6. Every PLANNED cell traces to a specific SP7 task (Phase A harnesses, Phase B HelixQA bump, Phase C sync-channel, Phase D code-review gate, Phase E guards); nothing here is claimed done that the tree does not show.

Sources verified 2026-06-10

- Plan: docs/superpowers/specs/plans/2026-06-10-SP7-testing-qa-plan.md (§1, §2, §7).
- Live tree: helix_code/tests/ (present: automation, e2e, integration, performance, regression, security, stresschaos, unit; **ABSENT**: ddos, scaling, ux, ui), helix_code/tests/stresschaos/stresschaos_meta_test.go (present), helix_code/tests/regression/ (critical_paths_test.go, server_timeout_test.go only), helix_code/internal/helixqa/wrapper.go (present), submodules/helix_qa pin v4.0.0-393-g4d2dcb2, submodules/helix_agent/internal/handlers/completion.go:406 (hardcoded list present), helix_code/internal/verifier/adapters.go:175 (GetMinAcceptableScore present, GetWorkingModels ABSENT).